함수

컴퓨터학부 박한영

목차

01
함수

02
정적 변수

03
재귀

함수

- 프로그래밍에서 함수(function)란 하나의 특별한 목적의 작업을 수행하기 위해 독립적으로 설계된 프로그램 코드의 집합이다.

C언어에서 함수는 표준 함수와 사용자 정의 함수로 구분할 수 있다.

- 표준함수(라이브러리 함수): C에서 기본적으로 제공하는 함수 (stdio.h, stdlib.h, string.h, …)
- 사용자 정의 함수: 사용자가 직접 정의해서 사용하는 함수. 우리가 지금까지 사용해왔던 main()함수도 이에 해당됨

함수

<함수의 필요성>

- 프로그램을 기능별로 나누어 구조적으로 작성할 수 있다.
- 코드의 가독성과 유지보수성이 향상됨
- 여러 사람이 기능을 분담하여 협업하기 쉬워진다.
- 반복되는 코드를 재사용하여 코드 중복을 줄일 수 있음
- 하나의 기능을 독립적으로 테스트하고 수정할 수 있음

함수 선언

```
return-type name(parameter-type1, type2, …); // 세미콜론은 필수!
```

-함수 선언은 함수의 반환형, 함수명, 매개변수 타입을 명시하는 것

-return-type은 함수가 반환하는 값의 자료형을 의미한다.

-name은 함수명이다.(프로그램이 함수를 호출할 때 필요한 식별자)

-parameter type은 매개변수의 타입을 명시한다.

-반환할 타입이 없는 경우 return type에 void라고 작성한다.

함수 정의

```
return-type name(parameter-type1 name1, type2 name2, …) {
        // 내부 구현
        return value;
}
```

-함수 정의는 함수의 실제 코드를 작성하고 기능을 구현하는 것

-return-type은 함수가 반환하는 값의 자료형을 의미한다.

-parameter type 뒤에는 매개변수의 변수명을 반드시 명시해야 한다.

-중괄호 내부에는 함수가 실제로 수행할 코드가 작성된다.

-return은 함수의 결과값을 호출한 곳으로 반환한다. (단, void의 경우 return x)

함수 반환

```
return-type name(parameter-type1 name1, type2 name2, …) {
        return value;
        // return; <- void type의 경우
}
```

-return은 함수의 실행을 종료하고 호출한 곳으로 돌아감

-return 뒤에 반환값과 같은 타입의 값을 작성해야 함.

-void 타입의 경우 return; 만 작성하거나 생략할 수 있음

함수 호출 과정

③ sum 함수 호출
(main 함수가 호출)

① main 함수 시작
(운영체제가 호출)

```
1   #include <stdio.h>
2   int sum(int x, int y){
3       ...
4       return tmp;
5   }
6
7   int main(void){
8
9       result = sum(a, b);
10      ...
11      return 0;
12  }
```

④ sum 함수 수행

② main 함수 수행

⑤ sum 함수 반환

⑥ 나머지 main 함수 수행

함수 선언 시 주의사항

변수를 사용하기 전에 선언하듯이 함수도 사용하기 전에 선언해야 함

- C언어는 단방향 컴파일러를 사용해 위에서부터 아래로 컴파일
- 만약 미리 선언되어 있지 않은 경우, 컴파일 에러가 발생!

```c
#include <stdio.h>

int main() {
    int a, b;
    sum(a, b); // error!
}

int sum(int x, int y) {
    return x + y;
}
```

함수명은 간결하고 의미가 보이도록 짓기

- 함수 명명 규칙은 변수와 동일(영문자, 숫자, 밑줄로 구성, 공백 x 등등…)
- 이름이 너무 길어질 경우, 코드 작성 시 어려움이 있음
- 함수명을 통해 어떤 함수인지 알 수 있으면 사용하기 편리함

함수 정의 시 주의사항

들여쓰기를 잘하기

- C언어는 파이썬과 다르게 들여쓰기로 인한 문법적 제약은 없음
- 그러나 가독성을 위해서 적절히 코드 블록 단위로 활용하는 것이 중요함

```
int main(){int a,b;scanf("%d%d",&a,&b);sum(a, b);return 0;}
```

공백을 적절히 이용하기

- 코드의 가독성을 위해서 사용
- 공백은 컴파일 과정에서 모두 사라지므로 신경 x

call by value

```
int main() {
    int a, b;
    sum(a, b);
}
```

함수 호출

a, b가 x, y로 복사

```
int sum(int x, int y) {
    return x + y;
}
```

- 복사의 의미
- a와 b를 함수 sum의 매개변수 자리에 넣음
- 이 변수를 sum에서 사용하는 것이 아니라, a와 b에서 저장된 값만 복사해서 사용

call by address

```c
int main() {
    int a, b;
    sum(&a, &b);
}
```

함수 호출

a, b의 주소가 x, y로 전달

```c
int sum(int *x, int *y) {
    return *x + *y;
}
```

- 변수의 주소를 전달
- a와 b의 주소를 함수 sum의 매개변수로 전달
- 원본 변수에 직접 접근할 수 있으므로, 함수 안에서 값을 변경할 수 있음
- 포인터 강의에서 자세히 다룰 예정(지금은 참고만!)

메모리 영역

- 변수가 저장되는 메모리 영역에 따라 아래와 같이 구분할 수 있음.

메모리 영역

코드 영역(Code Segment)

- 실행할 프로그램의 코드가 저장되는 영역
- text segment라고도 부름
- CPU는 code segment에 저장된 명령어를 하나 씩 처리함

데이터 영역(Code Segment)

- 프로그램의 전역 변수와 정적 변수가 저장되는 곳
- 프로그램의 시작과 함께 할당되며, 프로그램이 종료되면 소멸함

메모리 영역

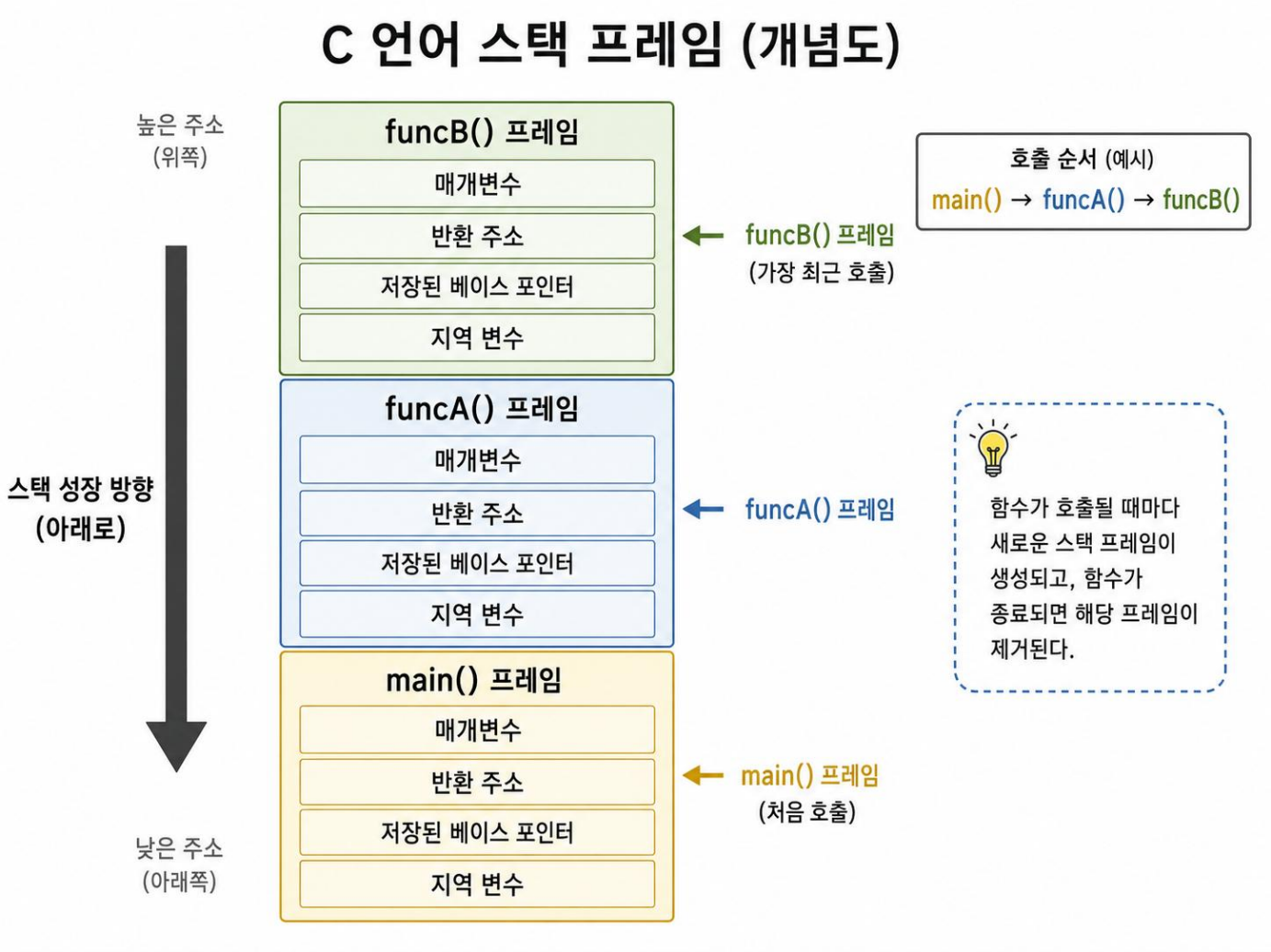
스택 영역

- 지역 변수와 함수의 매개 변수가 저장되는 영역
- 스택 영역은 함수의 호출과 함께 할당되며, 함수의 호출이 완료되면 소멸됨
- 스택 영역에 저장되는 함수의 호출 정보를 스택 프레임이라고 함

힙 영역

- 사용자가 직접 관리하는 메모리 영역
- 동적 메모리가 이곳에서 할당되고 해제됨
- 메모리 상위 주소에서 하위 주소의 방향으로 할당

스택 프레임


C 언어 스택 프레임 (개념도)
높은 주소
(위쪽)
스택 성장 방향
(아래로)
낮은 주소
(아래쪽)
funcB() 프레임
매개변수
반환 주소
저장된 베이스 포인터
지역 변수
funcA() 프레임
매개변수
반환 주소
저장된 베이스 포인터
지역 변수
main() 프레임
매개변수
반환 주소
저장된 베이스 포인터
지역 변수
funcB() 프레임
(가장 최근 호출)
funcA() 프레임
main() 프레임
(처음 호출)
호출 순서 (예시)
main() → funcA() → funcB()
함수가 호출될 때마다
새로운 스택 프레임이
생성되고, 함수가
종료되면 해당 프레임이
제거된다.

지역변수와 전역변수

지역 변수

- 한 블록 안에서만 사용 가능한 변수(블록은 중괄호{}로 묶인 범위)
- 지역 변수들은 블록을 벗어나면 소멸됨
- 스택 영역에 저장됨

```c
#include <stdio.h>

int global;

int main() {
    int local;
}
```

전역 영역

- 사용자가 직접 관리하는 메모리 영역
- 동적 메모리가 이곳에서 할당되고 해제됨
- 메모리 상위 주소에서 하위 주소의 방향으로 할당

정적 변수

-static 키워드를 사용해 선언함

-프로그램이 시작할 때 생성되어 끝날 때까지 존재함

-선언 시 자동으로 0으로 초기화됨

-함수 내부에서 선언 시 함수가 여러 번 호출되어도 초기화는 한 번만 실행

```
static int a; // 전역 static
static void function() { // static 함수
    static int i; // 지역 static
}
```

정적 변수

- 지역 변수의 경우 범위는 코드 블록 내부이지만, 수명은 프로그램과 동일
- 전역 변수의 경우 해당 파일 내부에서만 사용 가능(다른 곳에서 접근 불가)
- 함수도 해당 파일 내부에서만 사용 가능(내부 보조 함수에서 주로 사용)

```c
static int a;
static void function();
```

<header.h>

```c
int main() {
    a += 3;
    function();
}
```

<main.c>

재귀(설명 x)

재귀를 사용하는 이유: 변수 사용을 줄이고, 알고리즘에 따라 가독성을 높임

```c
/* 반복문으로 구현한 팩토리얼 함수 */
int factorial(int num) {
    int result = 1;
    for(int i = 1; i <= num; i++)
        result *= i;
    return result;
}
```

```c
/* 재귀로 구현한 팩토리얼 함수 */
int factorial(int num) {
    if(num == 1)
        return 1;
    return num * factorial(num - 1);
}
```

<재귀와 반복의 차이점>

- 재귀는 스스로를 호출
- 무한 반복이 발생하는 경우 재귀는 CPU 크래시를 초래할 수 있지만, 반복은 메모리가 부족할 때가 되면 멈춤

꼬리재귀(설명 x)

꼬리재귀: 재귀 호출이 다른 연산을 요구하지 않음 (=함수 자신만 재호출)

```c
/* 재귀로 구현한 팩토리얼 함수 */
int factorial(int num) {
    if(num == 1)
        return 1;
    return num * factorial(num - 1);
}
```

```c
/* 꼬리재귀로 구현한 팩토리얼 함수 */
int factorial(int num, int result) {
    if(num == 1)
        return result;
    return factorial(num - 1, result * num);
}
```

<꼬리재귀를 사용하는 이유>

- 스택 메모리를 절약할 수 있음
- 일부 컴파일러는 꼬리재귀에 대해 내부적으로 최적화 가능

-> 반복문과 유사하게 바꿀 수 있음!

THANK YOU!